
Real-Time & Advance Booking Pricing Architecture

A section-by-section response to the Real-Time & Advance Booking Pricing Architecture: what the engine already does, the Advance Booking Discount that was collected and never given, and the two decisions that are yours.

Aajoo Homes — Pricing Architecture

Prepared 6 September 2026 by the Zyphex Tech development team

Verified against the live development backend and database

1. Response summary

This answers **Aajoo Homes — Real-Time & Advance Booking Pricing Architecture**, section by section, in the order the document sets them out.

The document describes an engine that is largely already built. The three-tier grid, the composite Night/Week/Month calculation, the hidden floor, the auto-accept rule and the escalation to the host all shipped with W2 and are unit-tested. Running the document's own worked example through the live engine returns the document's own numbers — Minimum ₹45,000, Ideal ₹53,000, Maximum ₹62,000 — without anything being adjusted to make it do so.

Three things in it were not true, and are now.

The first is the one that matters. **§10–§12, the Advance Booking Discount, was not implemented at all.** The host was asked for a percentage in Step 4 of the listing wizard, the answer was stored, and the pricing resolver read it into the rule object — where nothing consumed it. Five live listings carry a discount today. Not one guest has ever been shown one, and not one host has ever given one away. It is implemented, deployed and verified on the live site as of today.

The second is **§15.** "The frontend should never independently calculate the final booking price" was half true: the website and the Android app each kept their own copy of the rate card. On short stays the three implementations agreed. On long stays the app's did not — it smoothed a weekly price across the nights while the server composes whole weeks plus remainder nights. Both clients now ask the backend and render its answer.

The third is a leak. `{min_price}` **was one of the variables an administrator could put in a property's page title or meta description.** §6 and §17 rule 4 make "the guest never sees the minimum" non-negotiable. One title pattern would have printed a host's negotiation floor into a page title and let Google index it. No live template used it; the variable is gone and a test now guards every door that number could leave by.

Two questions at the end need your decision rather than our judgement.

Response at a glance	
Sections answered	19 of 19
Already implemented, verified against the live system	14
Implemented during this response	3
Needs your decision	2
Backend	<code>11482d7</code> , <code>985fe68</code> , <code>b4dc3f1</code>
Website	<code>0753ccf</code> , <code>6cd929d</code>
Android app	<code>a88b106</code> , <code>70d0176</code> — build 32
Backend test suite	65/65 files pass
New tests written for this document	16 (<code>advanceDiscount</code> , <code>internalPriceNeverPublic</code>)

2. §1–§4 — Source of truth, host pricing structure, the meaning of the three prices

Status: already implemented.

The Real-Time Negotiation Engine specification v1.0 remains the locked foundation; nothing in this response changes a rule in it.

The listing wizard collects a Minimum, an Ideal and a Maximum for each of the three periods — nine figures per listing. They are stored in `property_pricing` (week and month) and on the listing row (night), and resolved into a single rule object by one function, `nightlyRates.pricingRuleFor`, which every pricing path calls.

The engine computes all three totals for every stay. The guest-facing block carries only the Maximum; the Minimum and Ideal totals are returned under a separate `internal` key that feeds the negotiation engine and is never serialised into a guest response. `tests/pricingQuote.test.js` fails the build if the quote controller so much as mentions that key, and `tests/pricingGrid.test.js` (17 cases) covers the grid itself.

Where a host has left a tier blank, it is derived from the period Maximum by the same ratio their nightly figures imply, rather than left null — so a partially-filled rate card still produces a coherent floor instead of switching negotiation off by accident.

3. §5 — The 12-night worked example

Status: already implemented. The engine returns the document's numbers exactly.

The stay is composed of whole months first, then whole weeks, then remainder nights, each charged at the host's stated figure for that period. Twelve nights is one week plus five nights.

Run against the document's rate card:

	Week	5 nights	Total	Document
Minimum	₹25,000	$5 \times ₹4,000 = ₹20,000$	₹45,000	₹45,000
Ideal	₹28,000	$5 \times ₹5,000 = ₹25,000$	₹53,000	₹53,000
Maximum / List	₹32,000	$5 \times ₹6,000 = ₹30,000$	₹62,000	₹62,000

One guard the document does not mention and the engine enforces anyway: a composed total is never allowed to exceed what the same nights would cost one at a time. A host who set a weekly price above seven times their nightly rate would otherwise charge a guest *more* for staying longer.

4. §6 — What the customer sees

Status: already implemented, and one leak closed this review.

The guest sees the Maximum/List total and never the other two. Verified at four doors:

Door	State
<code>POST /pricing/quote</code>	Field-by-field response; the <code>internal</code> block is never spread in. Guarded by a source test.
<code>GET /properties/:id</code>	<code>property_mini_price</code> and <code>property_ideal_price</code> are stripped for anyone who is not the listing's owner.
The quote object itself	No key beginning <code>min</code> or <code>ideal</code> exists in the guest block. Asserted over every key, not a fixed list.
SEO page titles and meta descriptions	Was leaking. Now closed.

The fourth was real. The SEO module offers an administrator a vocabulary of variables to write title and description patterns with — `{property_name}`, `{city}`, `{price}` — and `{min_price}` was among them. A single pattern would have put a host's negotiation floor into a page title, and a page title is the one place a leak cannot be taken back once Google has it. Neither of the two seeded templates used it and nothing published carried it, so no floor was actually exposed. The variable is removed, the bulk query no longer even selects the column, and typing `{min_price}` now returns "not a variable" rather than rendering silently.

`tests/internalPriceNeverPublic.test.js` (6 cases) covers all four doors, and asserts the vocabulary actually loaded first — an empty list would otherwise pass every check by proving nothing.

5. §7, §8 — Real-time booking and the below-minimum offer

Status: already implemented.

One engine decides, and every entry point calls it — the website, the Android app's chat, and the WhatsApp bot. Before that consolidation an app guest who offered above the floor waited for a decision a website guest got instantly: the same offer on the same property, answered differently depending on which screen they were on.

Rule	Behaviour
Offer $\geq$ Minimum	Accepted automatically at the offered price , not at the floor. The host is notified that a booking was completed, not asked to approve it.
Offer $<$ Minimum	Sent to the host as a pending row with Accept / Counter / Decline. Never auto-accepted.
Offer $>$ List price	Refused outright — there is nothing to negotiate above the number on the screen.
No Minimum configured	Negotiation is off for that listing and the guest is told to book at the listed price (rule 5).

Counters run up to a fixed number of rounds each way. The original implementation allowed one round each and then stuck: a guest's only replies to a host's counter were yes and no.

Covered by `tests/negotiationEngine.test.js` (24 cases), `negotiationGuards`, `negotiationCounter` and `negotiationAutoCounter`.

6. §9 — Real-time booking payment

Status: already implemented.

An accepted price reaches checkout as a personal, time-limited coupon pinned to that guest, that property and those dates. Both routes to acceptance — automatic and host-approved — mint the same thing, so the two produce an identical checkout rather than two code paths that must be kept in step.

The booking is then created at the agreed price. It does not revert to the list price at any point, and the booking endpoint recomputes the expected figure independently and refuses anything that disagrees with it by more than a rupee.

7. §10, §11, §12 — Advance booking, its pricing and its UI

Status: NOT IMPLEMENTED. Built, deployed and verified live today.

This is the substantive gap. The host was asked for an advance-booking discount, the answer was written to `property_pricing.ppr_advance_discount`, and `pricingRuleFor` resolved it into `advanceDiscountPercent` on the rule object — which was then read by nothing. The column existed, the form field existed, the resolver existed. The arithmetic did not.

Five listings on the live database carry a percentage today, four at 10% and one at 20%.

What was built

The discount is applied inside `nightlyRates.quoteRange` — the one function both the quote endpoint and booking creation call — rather than in a controller. That placement is the point: it means the price a guest is shown and the price the server will accept cannot disagree about whether a discount applied, and it makes §12's ordering rule true without the deposit code needing to know this rule exists.

It reduces the room subtotal, which is how a negotiated offer and a coupon already behave. A larger party still pays its extra-guest charge in full.

A same-day stay is untouched, per §13: real-time stays are priced by negotiation, and stacking a discount on a floor the host is negotiating against would move that floor without telling them.

Verified on the live site

Property 29290, twelve nights from 16 September, 10% advance discount. The figures below are read from the deployed website, not from a test:

Line	Shown
₹8,000 × 12 nights	₹95,000
Advance booking discount (–10%)	– ₹9,500
Taxes & GST (18%)	₹15,390
Total	₹1,00,890
Pay 10% now	₹10,089
Remaining before check-in	₹90,801

Ten per cent of the *undiscounted* total would have been ₹11,210. The gap between that and ₹10,089 is §12's ordering rule, visible on screen.

The same three lines appear on the property page, the review page and the payment page.

The part that was not about display

The checkout draft used to carry the browser's own room subtotal. Left that way, the discount would have shown on the property page and then been refused at checkout — the booking endpoint recomputes the same figure and rejects anything more than a rupee off, so **every discounted listing would have become unbookable**. And had it been accepted, the 10% deposit would have been taken on the larger number: §12's rule, inverted. The draft now carries the server's figure. The original price is re-derived for display only; nothing subtracts the discount twice.

`tests/advanceDiscount.test.js` (10 cases) pins the document's example — ₹62,000 → ₹55,800 — the ordering rule, the real-time exemption, and the rule that the discount never moves the negotiation floor.

8. §13 — Real-time vs advance

Status: already implemented, as a server-side rule rather than a UI convention.

Feature	Document	Implementation
Booking window	Current vs future	A stay starting today is real-time; tomorrow or later is advance. Decided in one place, in Indian Standard Time, and used by every surface.
Base pricing	Night / Week / Month	Same engine for both.
Public starting price	Maximum/List	Both.
Ideal / Minimum	Internal	Both. Never serialised.
Negotiation	Yes / No	The offer endpoint refuses an offer on a stay that does not start today.
Discount	Negotiated price / Advance Booking Discount	As above. The two cannot both apply to one stay.

Feature	Document	Implementation
Host decision	Only below minimum / none	As above.
Payment	100% / 10% advance	<code>payMode: "deposit"</code> is refused on a same-day stay, with an explanation. The deposit option is not offered on one either.

The deposit exists to hold a date that is still some way off; a guest arriving this afternoon has nothing to hold.

9. §14 — Complete system flow

Status: both flows match, with one presentational difference — see section 12 below.

The real-time flow runs exactly as drawn: dates → nights → three totals → show Maximum → offer → compare against Minimum → auto-accept or escalate → agreed price → 100% payment.

The advance flow now runs exactly as drawn too, which it did not this morning: dates → nights → three totals internally → Maximum as the public price → apply the discount → show original, discount and final → 10% of the final → confirmed.

10. §15 — One source of truth

Status: was half true. Now true on both clients.

The backend has had a single pricing engine since W2, and one public endpoint — `POST /pricing/quote` — that returns the full breakdown. What the document asks for is that the clients *use* it.

The website already called it, but the figure it handed to checkout was still its own. That is now the server's.

The Android app did not call it at all. It carried a copy of the rate card and worked the room charge out on the device. For a short stay the two agreed. For a long one they did not: the app divided a weekly price across the nights, while the server charges whole weeks at the host's weekly figure plus the remainder at the nightly rate. The advance discount would have made that divergence total — the app could not have booked any of the five discounted listings, because the price it sent would have been refused.

The app now asks the server and renders the answer, with the local arithmetic kept only as a fallback so a quote that has not yet arrived never blanks out a price a guest is reading. **Android build 32** carries this.

11. §16, §17, §18, §19 — Validation, the non-negotiable rules, the final model

Status: already implemented.

On §16: most of the list is not *validated* from the client — it is recomputed on the server and the client's figure is compared against it. That is the stronger form. Dates are refused unless they are canonical `DD-MM-YYYY`; nights, the three period prices, the three tiers, the discount eligibility and amount, the final price, the advance and the remaining balance are all derived server-side. Availability is re-checked inside the booking transaction, with the property serialised so two guests cannot both pass an overlap test and both insert.

On §17, all ten rules hold, and each is a test rather than a convention. Every negotiation decision is written to `tbl_negotiation_log` with the pricing snapshot, the round, the channel and the outcome — including the automatic acceptances, which is the case most likely to be left out. The decision function itself is pure: it takes numbers and returns an action, touching neither the database nor the clock.

On §18 and §19: one engine, two experiences, which is what is deployed.

12. Two questions for you

Neither is a defect. Both are places where the document is silent, or where following it literally would break something else, and the answer is yours rather than ours.

The 10% advance: on the price, or on the bill?

§12 works the example without tax: ₹55,800 → pay ₹5,580, remaining ₹50,220. The live platform charges GST on top of the room price, so 10% of what the guest actually owes is 10% of the tax-inclusive total.

We have kept the tax-inclusive reading — in the live example above, ₹10,089 of a ₹1,00,890 bill — because the "remaining" figure then settles the whole bill, and a deposit taken on a pre-tax number would leave the guest owing more than the document's arithmetic implies. **If you want the 10% taken on the pre-tax price instead, it is a one-line change** and we will make it. It does not affect the ordering rule either way.

Should the guest's offer be entered as a stay total?

§7 states the negotiation in stay totals: list ₹62,000, guest offers ₹50,000. The offer box asks for a figure **per night**, which is what the host's screen, the app, the chat bot and the accepted-price coupon all speak. The decision reached is identical — the engine converts the stay floor to a per-night floor and compares — but the guest is typing a different unit from the one the document uses.

We have added the stay total to the offer box, beside the stay's list total, so the guest now compares the same two numbers the document compares. **Changing the input itself to a stay total is a larger change**: the host would still be shown a per-night figure when deciding, unless every negotiation surface moves together. Tell us if you want that and we will plan it as its own piece of work rather than fold it into this one.

13. What changed today

Change	Where
Advance Booking Discount applied inside the shared pricing function	<code>utils/nightlyRates.js</code>
Discount, original price and percentage returned by the quote endpoint	<code>controllers/pricing.controller.js</code>
<code>{min_price}</code> removed from the SEO variable vocabulary; column no longer selected	<code>utils/seoTemplate.js</code> , <code>controllers/adminSeoBulk.controller.js</code>
Three-line breakdown on the property, review and payment pages	Website
Checkout draft carries the server's room subtotal, not the browser's	Website
Stay total shown beside the per-night offer	Website
App calls <code>/pricing/quote</code> ; local arithmetic becomes the fallback	Android, build 32
Three-line breakdown in the app's price panel	Android, build 32
Release script passes <code>-AllowTestPayments</code> through to the artifact verifier	Android build tooling
<code>tests/advanceDiscount.test.js</code> , <code>tests/internalPriceNeverPublic.test.js</code>	16 new cases

Everything above is deployed. The website and backend are live; the Android artifact is **Aajoo-Homes-v1.0.0-build32.apk**.